Python 101 – *Part 0*

2025.2

윤형기 (hky@openwith.net)

Intro

- Instructor 소개

- 필요성 (배경): Why 프로그래밍, 코딩?

https://coders.co.kr/3

컴퓨터와 프로그래밍 언어

- 컴퓨터와 사용자

분석자
(Data Scientists)

분석자
(Data Engieer)

분석자
(Analyst)

일반사용자

시스템관리자

소프트웨어 (시스템/응용)
(컴파일러, DBMS, 네트워크 SW, ...)

운영체제
(Linux, MS Windows, ...)

컴퓨터 H/W
(서버, PC, Mobile, IOT/센서)

- 프로그래밍 언어
  - 원하는 작업을 컴퓨터가 이해하도록 만들어 줌 (컴파일러 vs. Interpreter)
  - Assembly, C/C++, Java, …, Python (VHLL)

Python

- Python?
  - 범용의 프로그래밍 언어 (배우기 쉽고 분석 특화)
  - 오픈소스

	Python	R	Shell	C++
일반사항	General-Purpose	DSL	Linux/Unix	General-Purpose
장점	투명성 (.py), AI, 패키지/community	통계/시각화, 패키지/Bioconductor	사용 용이	성능, 확장성
단점	하위 호환성 (V.E.), 성능	패키지 in C/C++, Memory 한계	성능, LX only	Low-level

- Bio 분야에서의 Python
  - 유전체 데이터의 분석 (Biopython, scikit-learn, TensorFlow)
  - Drug Discovery & Personalized Medicine (AlphaFold, DeepChem, RDKit, …)
  - Medical Imaging & Diagnostics (OpenCV, TensorFlow, PyTorch)
- Python과 AI - Python as a ***King of AI***
  - AI = Data + Python "AI is just a Fancy Statistics"
    - Misconceptions of AI - Hollywood AI vs. Real AI
    - Real-World AI: ***Advanced data processing***, prediction models
  - 데이터 분석 절차
    - 데이터 수집 (실험, 센서, 데이터베이스)과 정제 (noise 제거, 결측치 보충, …)
    - 분석 (통계분석, 모델링, 시각화)
    - 활용: 예측 & 자동화

Let’s learn Python

- Python as a universal language of scientific computing.

회차	주요 내용
Part 0	▪ Let's learn Python!
Part 1	▪ Python 소개 ▪ 실습환경 설치
Part 2	▪ Python 데이터타입
Part 3	▪ 파일 이용 ▪ 흐름제어와 함수
Part 4	▪ Class와 OOP, Exception ▪ Module과 package 이용법
Part 5	▪ Standard library, Jupyter Notebook ▪ 분석 패키지 (numpy, 시각화, pandas 등)
Part 6	▪ 분석 개요, ▪ 기계학습과 딥러닝, AI

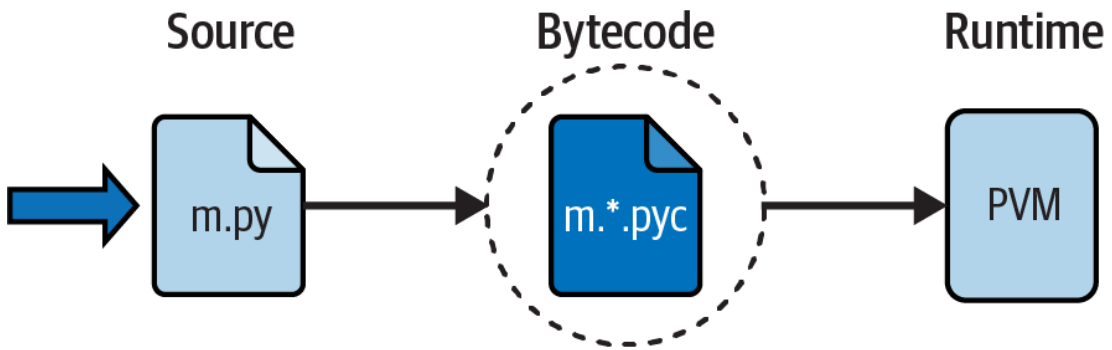
Python 101 – *Part 1*

2025.2

윤형기 (hky@openwith.net)

Python 특징

- 특징
  - A general-purpose programming language
  - Interpreted *vs. Compiled*
  - A very high-level language (VHLL)
  - Strongly-Typed, Dynamic Typing
- Python Implementations
  - CPython ; Classic Python (Python) = implementation of Python.



- 몇 가지 기본 용어 (*if needed*)
    - identifier 식별자 (이름)
    - keyword 프로그래밍 언어에서 미리 정해 놓은 명령어
    - parameter 함수나 객체의 정의를 위해 전달하는 값 (argument)
    - variable 변수 (변하는 값을 가지는 것)
    - literal 변하지 않는 값 (expressing a non-changing value)
    - operator 연산자
    - expression 표현식
    - statement 명령문.
    - class 객체를 생성하기 위한 틀 (template)
    - object 개발자가 만들어 내는 데이터 타입 (abstract data type)
    - Integer 정수
    - Floating-point 부동 소수점
    - String 문자열
- https://docs.python.org/3/glossary.html#glossary
  - https://docs.python.org/ko/3/glossary.html#glossary

실습

실습환경

- 개발도구 (partial)

	Py/IDLE	VS Code	vi	Spyder/ ipython	Anaconda	Eclipse	Pycharm	Atom
Windows								
Linux								

- 실습환경
  - Linux vs. MS Windows
  - 기타 : Raspberry Pi, Cloud

- Python Virtual Environments
  - Python version별로 독립된 (isolated) 가상환경 이용

pyenv	virtualenv	venv	pipenv	Conda

- 환경설치
  - python.org
  - Anaconda
    - Spyder
  - Visual Studio Code
  - PyCharm
  - Eclipse PyDev

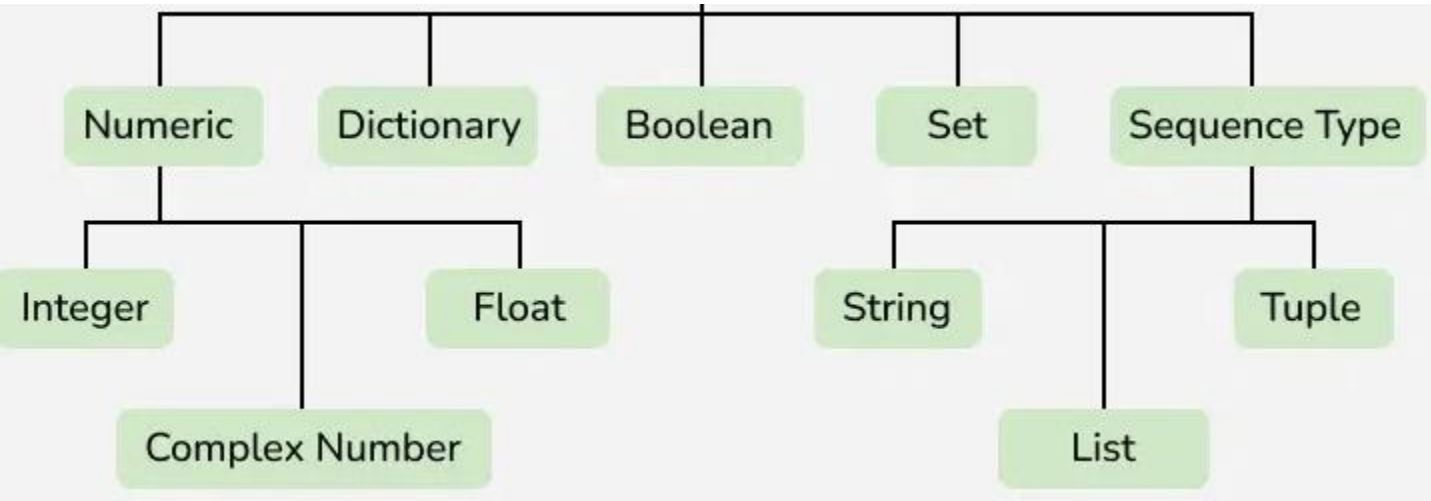
Python 101 – *Part 2*

2025.2

윤형기 (hky@openwith.net)

주요 데이터 타입

- Built-in Data Types



Text Type	str
Numeric	int, float, complex
Sequence	list, tuple, range
Mapping	dict
Set Types	set, frozenset
Boolean	bool
Binary	bytes, bytearray, memoryview

- User-defined Types
  - class 이용하여 생성

내장 (BUILT-IN) 데이터 타입

(1) Numbers

- Python에서의 4가지의 숫자 타입
  - Integers
    - 1, -3, 42, 355, 888888888888888, -7777777777
  - Floats
    - 3.0, 31e12, -6e-4
  - Complex numbers
    - 3 + 2j, -4- 2j, 4.2 + 6.3j
  - Booleans
    - True, False
  - Fraction (분수)와 Decimal
  - 숫자 데이터에는4칙연산과 ** (제곱) 및 % (나머지 계산) 이 가능

(2) 문자열 (Strings)

- = a sequence of characters
- 생성방법: 다양한 따옴표를 이용
  - single (' '), double (" "), triple single (''' '''), or triple double (""" """) quotation
  - 이렇게 만들어진 문자열에는 tab (₩t)과 줄바꿈 문자 (₩n)가 그대로 유지된다.

```
"A string in double quotes can contain 'single quote' characters."
'A string in single quotes can contain "double quote" characters.'
'''₩tA string which starts with a tab; ends with a newline character.₩n'''
"""This is a triple double quoted string, the only kind that can
    contain real newlines."""
```

- 특징
  - 다양한 기능 적용 가능
  - 단, immutable (= 문자열은 변경이 불가능)
    - + 연산자로 연결 시 원본 변경이 아닌 새로운 문자열이 만들어짐.

(3) Lists

- 다양한 형태로 이용이 가능한 sequence 타입의 데이터 구조체

```
[]
[1]
[1, 2, 3, 4, 5, 6, 7, 8, 12]
[1, "two", 3, 4.0, ["a", "b"], (5,6)]          1
```

  - List에는 서로 다른 형태의 데이터를 담을 수 있다.
  - indexing 또는 slicing
    - List 안의 항목은 순서에 따른 index를 가지므로 특정 위치/구간의 항목들을 뽑아낼 수 있다.

x=	[	"first" ,	"second" ,	"third" ,	"fourth"	]
Positive indices		0	1	2	3	
Negative indices		-4	-3	-2	-1	

- List Comprehension
  - List 를 한 줄로 만들어 사용하는 것
    - 예: 0~9의 숫자를 제곱

```
>>> squares = []
>>> for x in range(10):
...     squares.append(x**2)
...
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

    - List comprehension 이용

```
squares = [x**2 for x in range(10)]
```

(4) Tuples

- Tuples 은 list와 대부분 유사하지만 변경불가 (immutable)라는 점에서 큰 차이
- Tuple의 주된 사용처 중 하나는 dictionary에서의 key

```
()
(1,)
(1, 2, 3, 4, 5, 6, 7, 8, 12)
(1, "two", 3L, 4.0, ["a", "b"], (5, 6))
```

- List를 tuple로 변환하려면 tuple() 함수를 이용.
  - 역으로 list()를 통해 tuple 을 list로 변환.
- tuple에도 count 및 index등의 method가 제공됨.

```
>>> x = [1, 2, 3, 4]
>>> tuple(x)
(1, 2, 3, 4)

>>> x = (1, 2, 3, 4)
>>> list(x)
[1, 2, 3, 4]
```

(5) Dictionaries

- Key-Value Pair 의 데이터구조체.
- 다양한 method (예: clear, copy, get, items, keys, update, values).

```
>>> x = {1: "one", 2: "two"}
>>> x["first"] = "one"
>>> x[("Delorme", "Ryan", 1995)] = (1, 2, 3)
>>> list(x.keys())
['first', 2, 1, ('Delorme', 'Ryan', 1995)]
>>> x[1]
'one'
>>> x.get(1, "not available")
'one'
>>> x.get(4, "not available")
'not available'
```

- (유의)
  - Dictionary에서는 Key가 중심이 되기 때문에 변경이 불가능. 단, Value는 변경 가능.

(6) Sets

- 일종의 집합
  - 순서를 가지지 않는 객체들의 모음 (unordered collection of objects)
  - 유일성 (uniqueness)과 구성요소 (membership)의 개념을 중심으로 사용

```
>>> x = set([1, 2, 3, 1, 3, 5])
>>> x
{1, 2, 3, 5}
>>> 1 in x
True
>>> 4 in x
False
>>>
```

- 참고
  - “python tutorial” 로 검색

Python 101 – *Part 3*

2025.2

윤형기 (hky@openwith.net)

파일

파일의 이용

- 데이터 저장을 위한 것으로서 파일객체(file object)로서 이용
- 이용방법
  - File 타입; Text files, Binary files
  - File Modes: 파일을 열 때 수행 목적과 작업에 따라 mode를 지정 (r, w, a, …)

```
>>> f = open("myfile", "r")
>>> line1 = f.readline()
>>> line2 = f.readline()
>>> f.close()

>>> f = open("myfile", "w")
>>> f.write("First line with necessary newline character₩n")
44
>>> f.write("Second line to write to the file₩n")
33
>>> f.close()
```

```
>>> print(line1, line2)
First line with necessary newline character
Second line to write to the file
>>> import os
>>> print(os.getcwd())
c:₩My Documents₩test
>>> os.chdir(os.path.join("c:₩₩", "My Documents", "images"))
>>> filename = os.path.join("c:₩₩", "My Documents",
"test", "myfile")
>>> print(filename)
c:₩My Documents₩test₩myfile
>>> f = open(filename, "r")
>>> print(f.readline())
First line with necessary newline character
>>> f.close()
```

- 기타 ; 파일 종류에 따른 이용
  - csv, .json, .png, …

```
import csv
with open('Giants.csv', mode ='r')as file:
  csvFile = csv.reader(file)
  for lines in csvFile:
        print(lines)
```

- with 를 이용한 Context handling
  - 목적: 파일자원의 관리를 보다 안정적으로 하고자 함.
    - 파일 닫기 등 작업이 끝난 후 뒷정리 (clean-up)를 자동으로 해 준다.

```
filename = "myfile.txt"
with open(filename, "r") as f:
    for line in f:
        print(f)
```

흐름 제어 (CONTROL FLOW STRUCTURES)

Boolean values와 표현식 (expression)

- Python에서의 Boolean값 : False & True
  - False
    - False, 0 None, 비어 있는 (empty) 값 (예: list [] 또는 빈 문자열 "")
    - Booleans 표기에서 False는 0
  - True
    - 그 이외의 모든 것.
    - Booleans 표기에서 True는 1

- True or False 생성
  - 비교연산 (<, <=, ==, >, >=, !=, is, is not, in, not in) 내지
  - 논리연산 (and, not, or)

if-elif-else 문

- if/ elif/ else + True or False에 의해 흐름 제어

```
x = 5
if x < 5:
    y = -1
    z = 5
elif x > 5:
    y = 1
    z = 11
else:
    y = 0
    z = 10
print(x, y, z)
```

  – Python에서는 구간 (block)자정에 들여쓰기 (indentation)만 사용

while loop

- 제시된 조건이 충족되는 한 계속 반복.

```
u, v, x, y = 0, 0, 100, 30
while x > y:
    u = u + y
    x = x - y
    if x < y + 2:
        v = v + x
        x = 0
    else:
        v = v + y + 2
        x = x - y - 2
print(u, v)
```

for loop

- Python에서 for 는 사실상 foreach loop의 형태로 이용

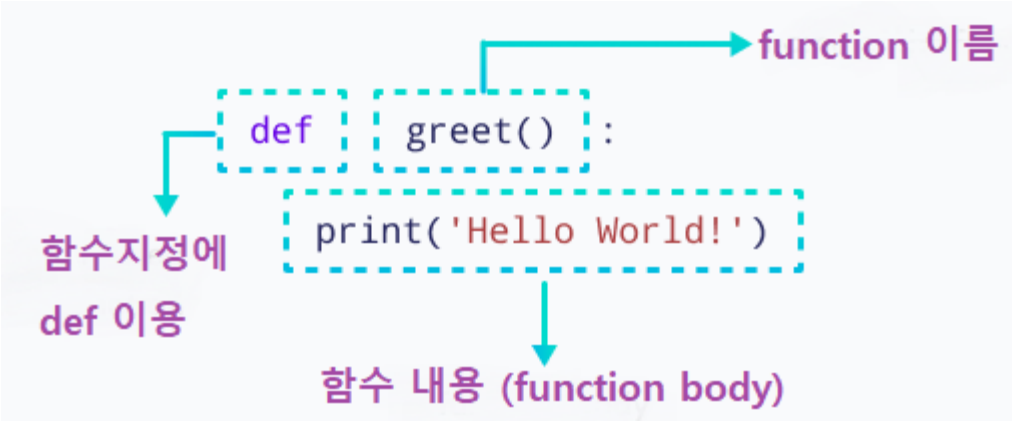
```
item_list = [3, "string1", 23, 14.0, "string2", 49, 64, 70]
for x in item_list:
    if not isinstance(x, int):
        continue
    if not x % 7:
        print("found an integer divisible by seven: %d" % x)
        break
```

  – 문장 중 continue는 그 이하의 부분을 처리하지 않고 다음 loop로 이동하여 실행하라는 의미.

함수

함수 작성

- def 문을 이용하여 정의 + return 문을 통해 반환 값 지정
  - return 문이 없으면 Python은 None 을 반환




```
>>> def greet():
...     print('Hello World!')
...
>>> greet()
Hello World!
```

- 함수에서의 인자 전달
  - 기본: positional argument ; 위치에 따라 대응되는 방식으로 인자를 전달
  - 기타: keyword 명시, 또는 가변 (variable) 개수 등 다양한 형태 가능

```
>>> def funct1(x, y, z):
...         value = x + 2*y + z**2
...         if value > 0:
...             return x + 2*y + z**2
...         else:
...             return 0
>>> u, v = 3, 4
>>> funct1(u, v, 2)
15
>>> funct1(u, z=v, y=2)
23
```

```
>>> def funct2(x, y=1, z=1):
...     return x + 2 * y + z ** 2
>>> funct2(3, z=4)
21
>>> def funct3(x, y=1, z=1, *tup):
...     print((x, y, z) + tup)
...
>>> funct3(2)
(2, 1, 1)
>>> funct3(1, 2, 3, 4, 5, 6, 7, 8, 9)
(1, 2, 3, 4, 5, 6, 7, 8, 9)
>>> def funct4(x, y=1, z=1, **kwargs):
...     print(x, y, z, kwargs)
>>> funct4(1, 2, m=5, n=9, z=3)
1 2 3 {'n': 9, 'm': 5}
```

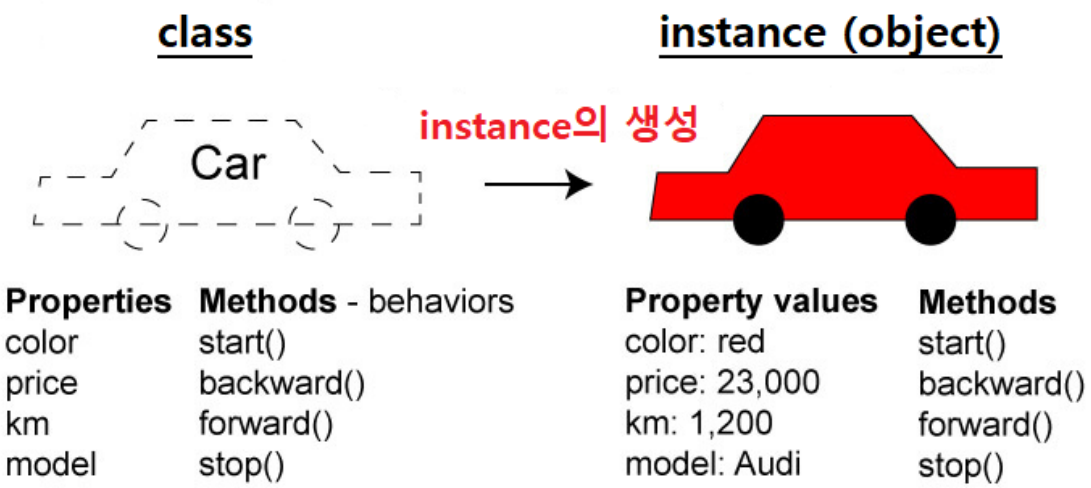
Python 101 – *Part 4*

2025.2

윤형기 (hky@openwith.net)

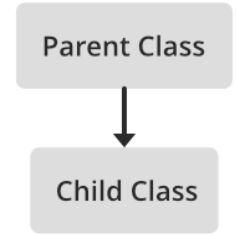
CLASS와
OBJECT-ORIENTED PROGRAMMING

- Class
  - = user-defined data type ; 일종의 "blueprint" (= constructor)

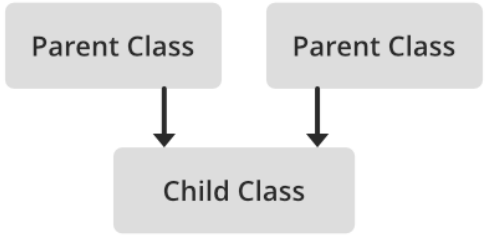


- OOP (Object-oriented Programming)
  - "objects" 개념을 중심으로 전개되는 프로그래밍 방식 (paradigm)
  - Encapsulation, Inheritance, Polymorphism

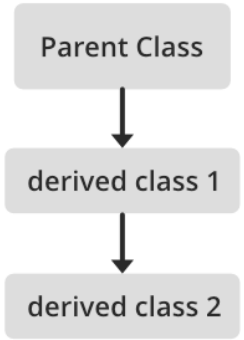
- Instance



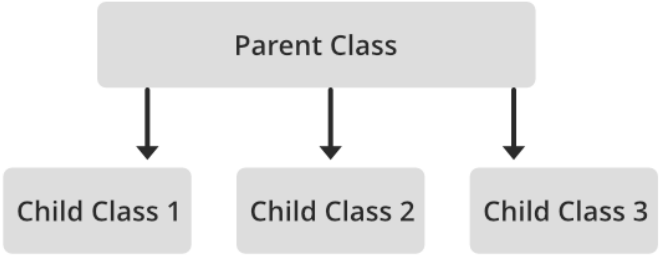
Single inheritance



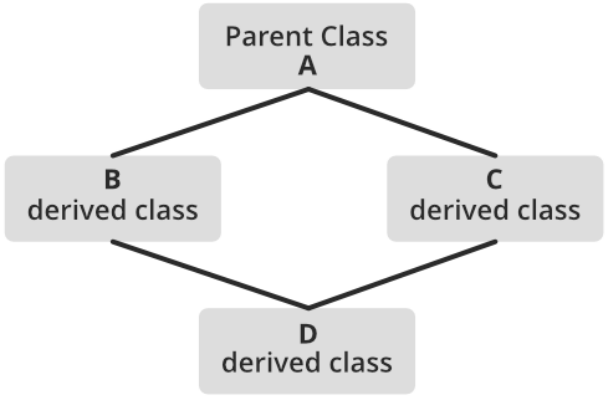
Multiple inheritance



Multilevel inheritance



Hierarchical inheritance



Hybrid Class

- class의 생성과 이용
  - class의 생성

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def greet(self):
    return f"My name is: {self.name}({self.age})"
```

```
p1 = Person("John", 36)
p1.greet()

p2 = Person("김영희", 28)
p2.greet()
```

  - class의 이용
    - import

car.py

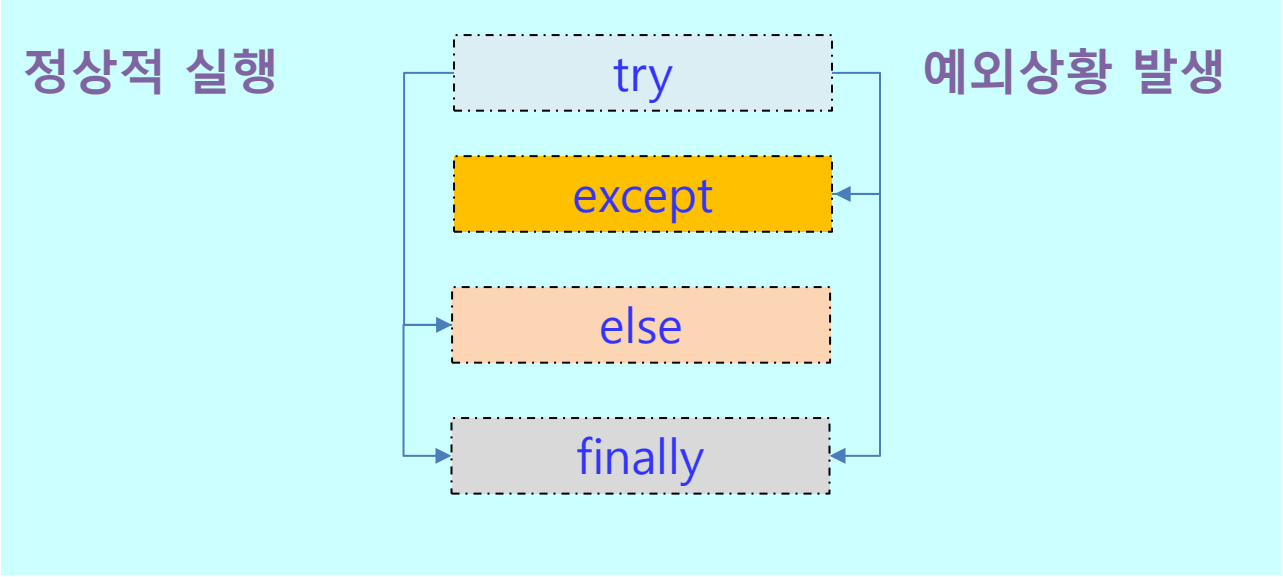
```
class Car:

    def __init__(self,make,model,year,color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color

    def drive(self):
        print("This "+self.model+" is driving")
    def stop(self):
        print("This "+self.model+" is stopped")
```

```
from car import Car

car_1 = Car("Chevy", "Corvette", 2024,
"black")
car_2 = Car("Ford", "Mustang", 2024,
"blue")

car_1.drive()
car_2.stop()
```

EXCEPTIONS

Exceptions

- exception handling - try-except-else-finally 문
  - 프로그램 수행 중 발생할 수 있는 예외적 (error)상황에 미리 대비하기 위한 것
  - 비정상 종료를 관리범위 내로 가져오려는 것
  - Class의 일종으로서 Python 내장된 것 이외에 사용자가 작성 가능



```
def divide(x, y):
    try:
        result = x // y
    except ZeroDivisionError:
        print("Error: You are dividing by zero")
    else:
        print("Your answer is :", result)
    finally:
        print('This is always executed')


# case 1
print(divide(1, 2))

# case 2
print(divide(1, 0))
```

MODULE과 PACKAGE 이용

Module

- Module
  - = (프로그램) 파일 (.py, C 프로그램, 상수 (constants) 등 가능)
  - 모듈에 담긴 내용을 다른 프로그램에서 이용하려면:
    - import *모듈이름*
    - from *모듈* import *함수/class이름*

[mymodule.py]

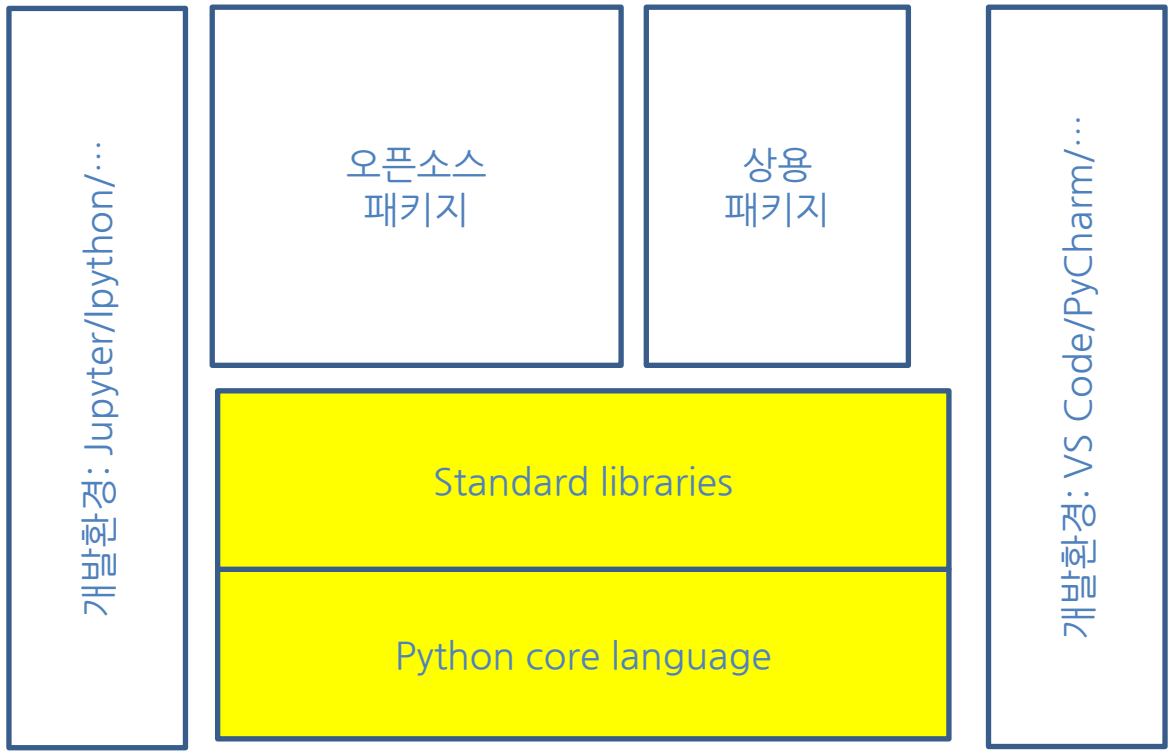
```
def greeting(name):
  print("Hello, " + name)
```

(이용)

```
import mymodule
mymodule.greeting("Jonathan")
```

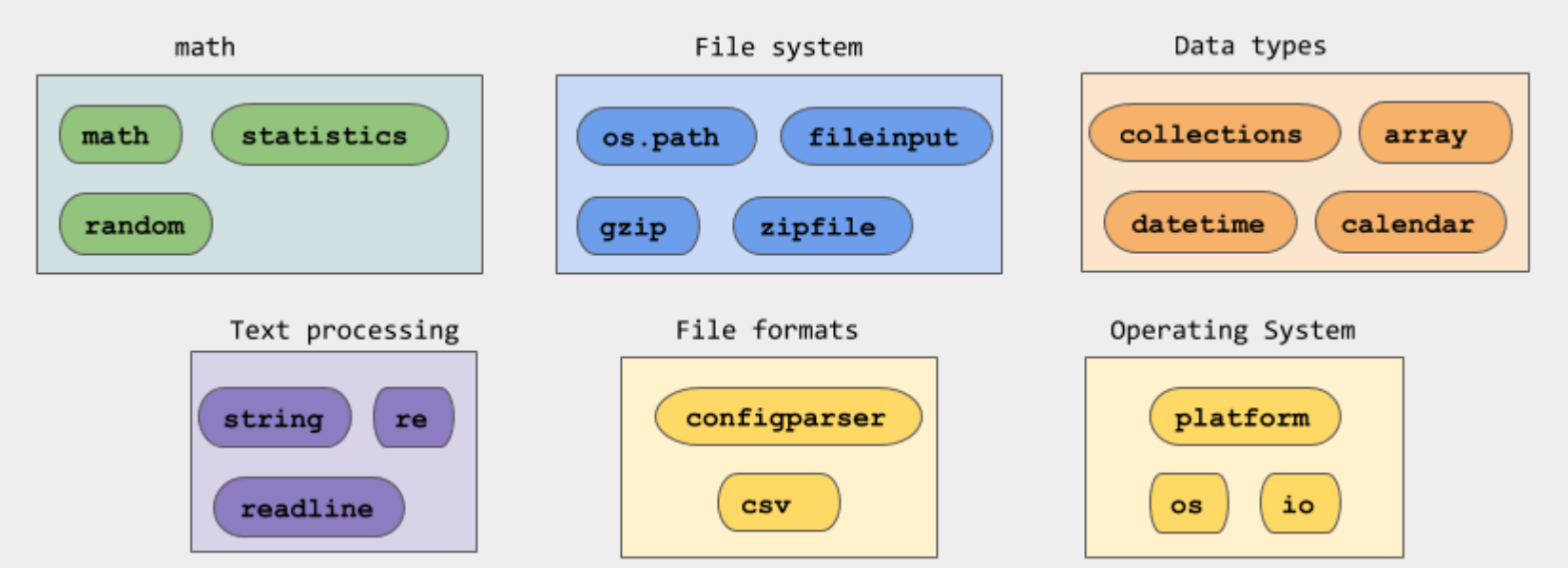
- Package
  - 여러 개의 module을 가진 folder 단위로 재사용
    - import *패키지이름*

Python Package Hierarchy



Standard Library

- *Batteries Included!*
  - https://docs.python.org/3/library/



- 예: collections 모듈
  - 다양한 형태의 container를 제공
    - Counters
    - NamedTuple
    - UserString …

    OrderedDict
    Deque

    DefaultDict
    UserDict

    ChainMap
    UserList
- 예: counter

```
from collections import Counter

my_list = ['B','B','A','B','C','A','B','B','A','C']
count_of_chars = Counter(my_list)
print(count_of_chars)

# with dictionary
print(Counter({'A':3, 'B':5, 'C':2}))

# with keyword arguments
print(Counter(A=3, B=5, C=2))
```

Python 101 – *Part 5*

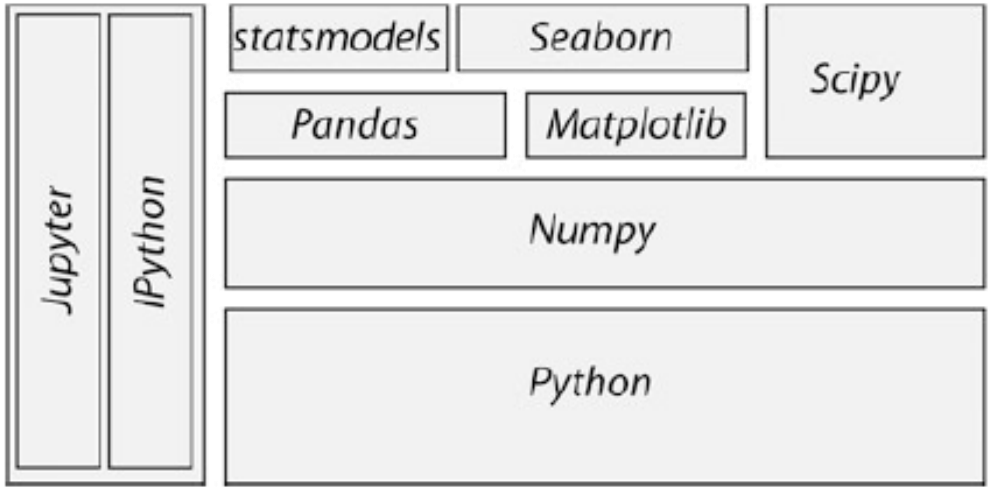
2025.2

윤형기 (hky@openwith.net)

PYTHON을 이용한 데이터 분석

Python 분석 패키지

- Python 분석 Libraries
  - Scipy
  - statsmodels
  - NumPy
  - Pandas
  - matplotlib
  - Ipython과 Jupyter



scipy와 sympy

- https://scipy.org/
  - Scientific computing 패키지
  - Task별 sub-package로 구성

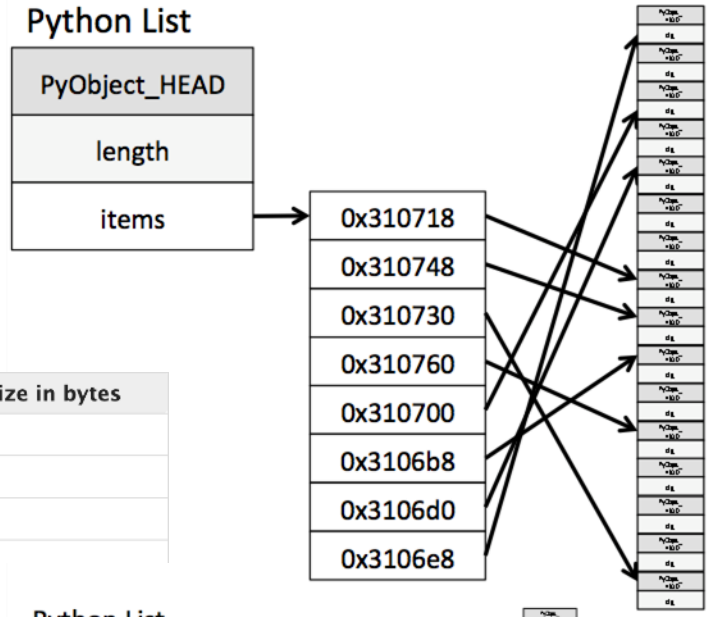
- https://www.sympy.org
  - symbolic mathematics

Subpackage 이름	내용
differentiate	Finite difference differentiation (유한차분, 미분)
fft	Fourier Transforms (푸리에 변환(變換))
integrate	Integration (scipy.integrate) (적분)
linalg	Linear Algebra (scipy.linalg)
ndimage	다차원 이미지 처리 (scipy.ndimage)
optimize	Optimization (scipy.optimize)
signal	Signal Processing (scipy.signal)
stats	Statistics (scipy.stats)
… https://docs.scipy.org/doc/scipy/tutorial/index.html#user-guide	

NUMPY

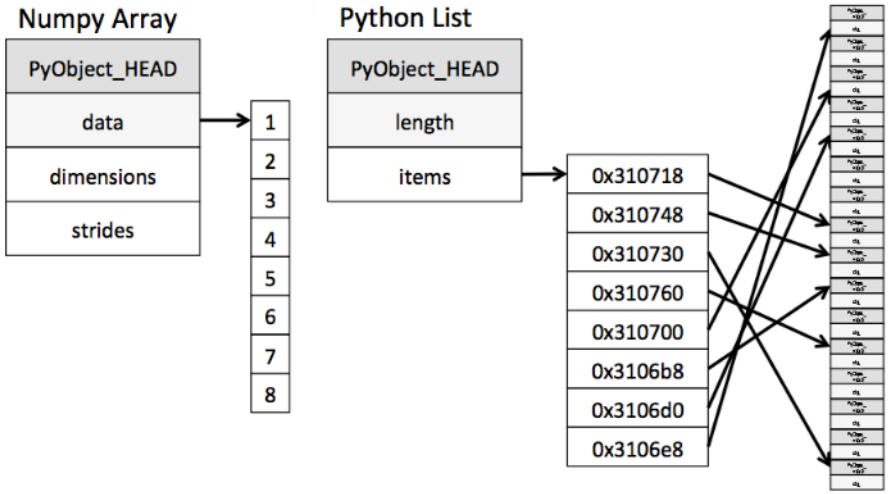
- 개요: ndarray 및 arrays 관련 함수
  - ndarray
    - 다차원 array object
    - ndarrays 생성 및 Array와 Scalar 간의 연산
    - Indexing 및 Slicing;
      - 기본 및 Boolean Indexing, Fancy Indexing, Transposing, Swapping Axes
  - Universal Functions: Fast Element-wise Array Functions
  - Loop-free programming with arrays
    - Conditional Logic을 Array 연산의 형식으로 표현
    - 수학/통계계산 Methods, Boolean Array를 위한 methods, Set Logic, …
  - 파일 입출력과 Arrays; Saving and Loading Text Files
- 활용: 거의 모든 데이터 분석의 기본이 됨 → Tensor

- Python 데이터 구조 vs. Numpy 데이터 구조
  - Python
    - list
    - array.array
    - …



Type code	C Type	Python Type	Minimum size in bytes
'b'	signed char	int	1
'B'	unsigned char	int	1
'u'	Py_UNICODE	Unicode character	2
'h'	signed short	int	2

  - NumPy ndarray
    - 고속처리를 위한 다양한 기법 적용
    - → tensor



시각화 패키지

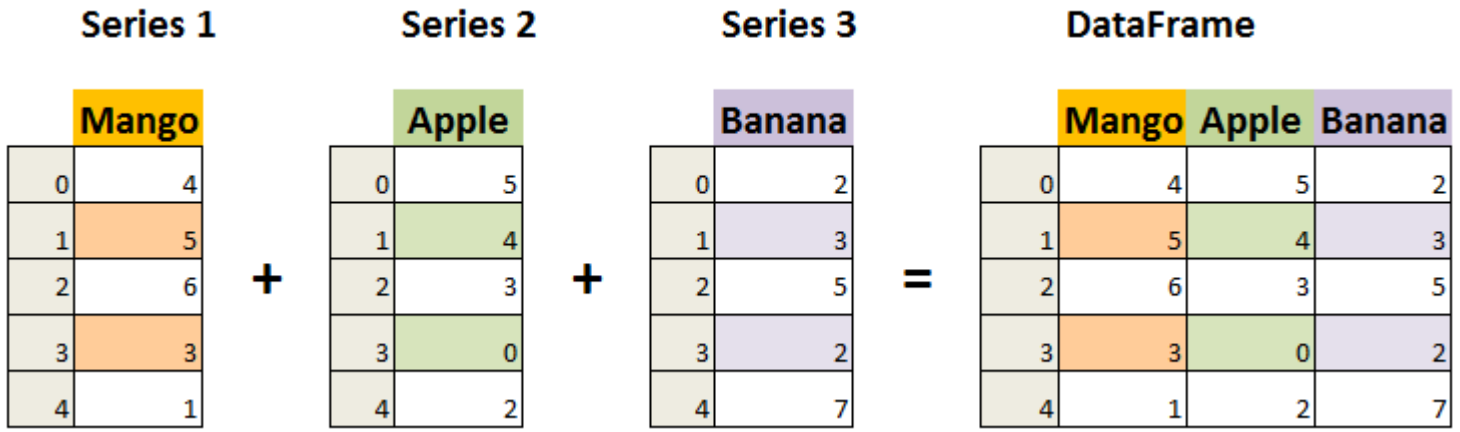
주요 시각화 패키지

- matplotlib
  - 개요
    - 대표적인 시각화 패키지 (http://matplotlib.org/gallery.html)
  - 사용방식
    - pylab (no longer recommended)
      - pyplot (plotting) 기능과 numpy 기능을 한 namespace에 통합
    - matplotlib.pyplot
      - 주로 non-interactive plotting용으로 이용 (i.e., scripting).
  - plotting 함수의 입력은 원칙적으로 np.array
- Seaborn
  - matplotlib 확장 패키지
- 기타: ggplot, Bokeh, …

PANDAS

pandas 기초

- 핵심개념
  - 분석 주요 기능을 담고 있는 패키지 ← R에서의 data.frame
    - 전처리/정제/통합/ … ; 데이터베이스 + Excel 기능을 python 명령으로 구현
      - » https://pandas.pydata.org/pandas-docs/stable/getting_started/comparison/comparison_with_sql.html
- pandas.core
  - GroupBy engine
  - Time series tools
  - Hierarchical indexing s
  - …
- pandas object
  - Series와 DataFrame



- 주요 기능
  - Label-based indexing/Reindexing, selection, and filtering
  - GroupBy: data set에 대한 aggregation과 transformation
  - Reshaping, pivot tables
  - Merge, concatenation
  - Sorting and ranking
  - Axis indexes with duplicate values
  - 결측 데이터의 처리 - Filtering Out 과 Filling in Missing Data
  - Time series 기능

- 참고자료:
  - https://numpy.org/learn/
  - https://jupyterlab.readthedocs.io/en/stable/user/interface.html
  - https://matplotlib.org/stable/tutorials/pyplot.html

  - https://lectures.scientific-python.org/

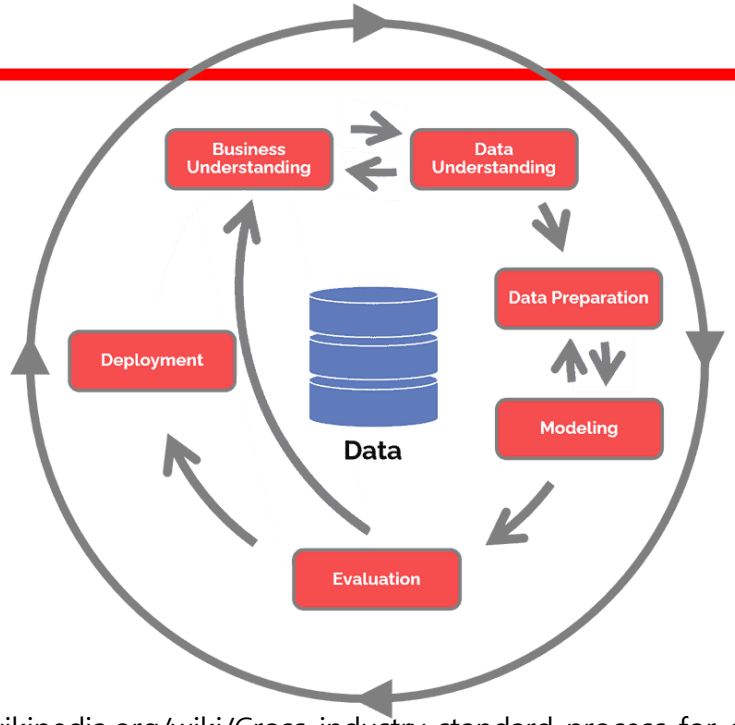
Python 101 – *Part 6*

2025.2

윤형기 (hky@openwith.net)

분석 개요

- 기계학습과 딥러닝 - 분석 프로세스



https://en.wikipedia.org/wiki/Cross-industry_standard_process_for_data_mining

① Business Understanding

- 업무 목표 수립
- 현재 상황 평가
- 마이닝 목표 수립
- 프로젝트 계획 수립

② Data Understanding

- 초기 데이터 수집
- 데이터 기술
- 데이터 탐색
- 데이터 품질 검증

③ Data Preparation

- 데이터 설정
- 데이터 선택
- 데이터 정제
- 데이터 생성
- 데이터 통합
- 데이터 형식 적용

④ Modeling

- 모델링 기법 선택
- 테스트 설계 생성
- 모델 생성
- 모델 평가

⑤ Evaluation

- 결과 평가
- 프로세스 재검토
- 향후 단계 결정

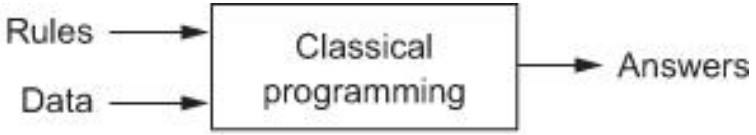
⑥ Deployment

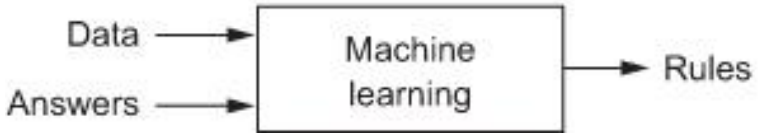
- 전개 계획 수립
- 모니터링/유지보수 계획 수립
- 최종 보고서 작성
- 프로젝트 재검토

기계학습

모델링

- 기계학습
  - 현재 및 과거 데이터를 분석하여 미래 이벤트를 예측 (predictive analysis)





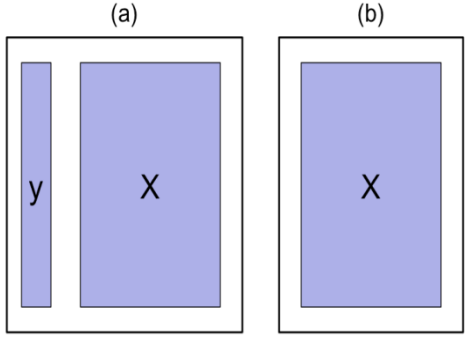
  - 모델링
    - f의 추정?

$$\hat{Y} = \hat{f}(X),$$

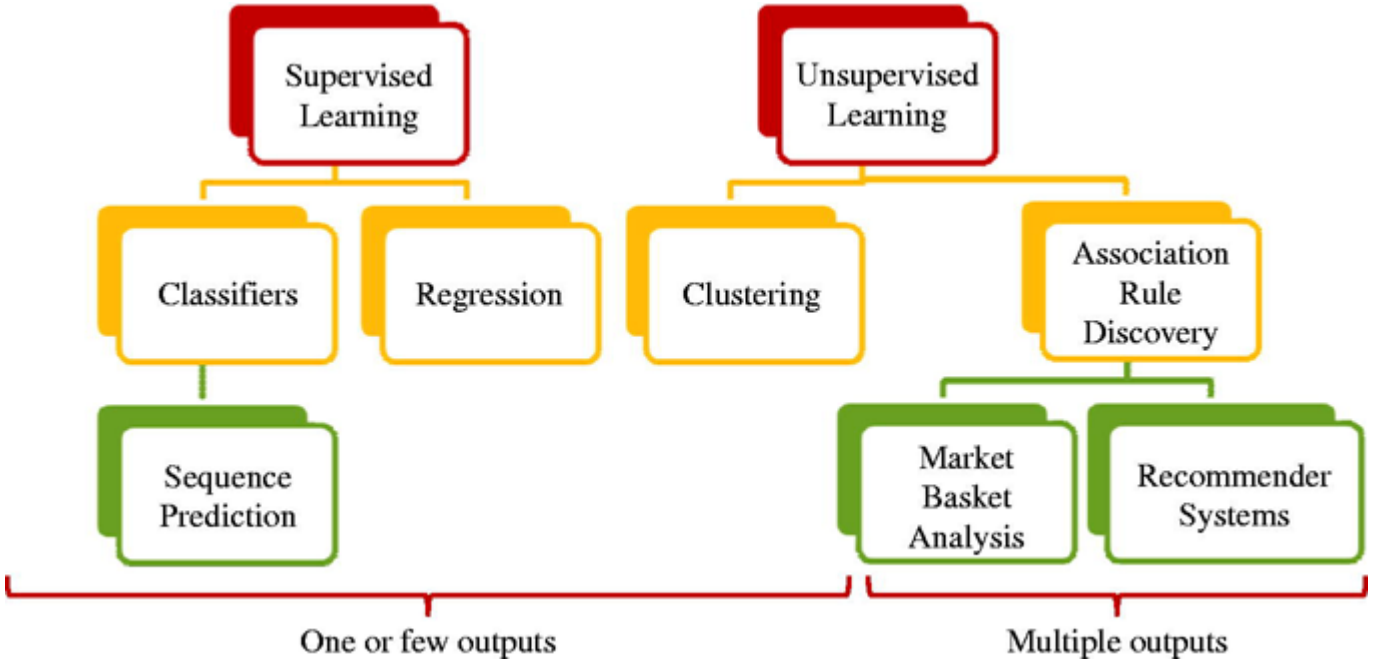
$$\begin{aligned} E(Y-\hat{Y})^2 &= E[f(X)+\epsilon-\hat{f}(X)]^2 \\ &= \underbrace{[f(X)-\hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\mathrm{Var}(\epsilon)}_{\text{Irreducible}} , \end{aligned}$$

지도학습 vs. 비지도학습

- 분류
  - 감독학습 (Supervised Learning)
  - 무감독학습 (Unsupervised Learning)
  - Semi-supervised Learning



Supervised Learning Unsupervised Learning



- 기계학습 주요 알고리즘

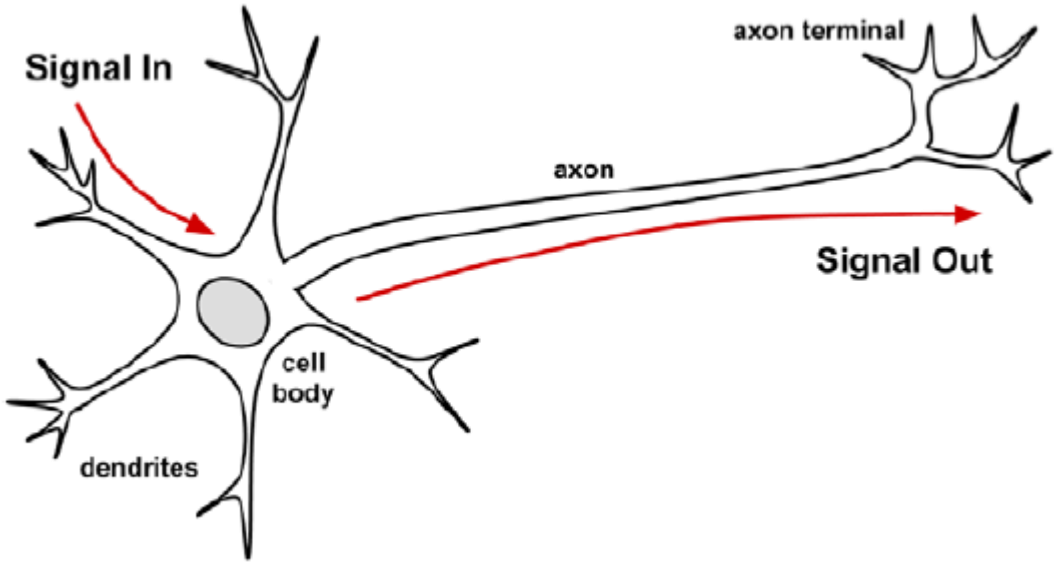
감독학습	무감독학습	기타
kNN (근접이웃)		
(Naïve) Bayes		
의사결정트리		
(Classif'n Rule L)		
선형회귀		
인공신경망 /딥러닝	**인공신경망 /딥러닝**	**인공신경망 /딥러닝**
SVM (support vector Machines)	AR (연관규칙 Association Rules)	
	K-means	
	..	차원축소, 행렬분해
		추천, Anomaly 탐지
		자연어처리

- Python의 기계학습 패ㅐ키지
  - https://scikit-learn.org
  - https://scikit-learn.org/1.4/tutorial/machine_learning_map/index.html

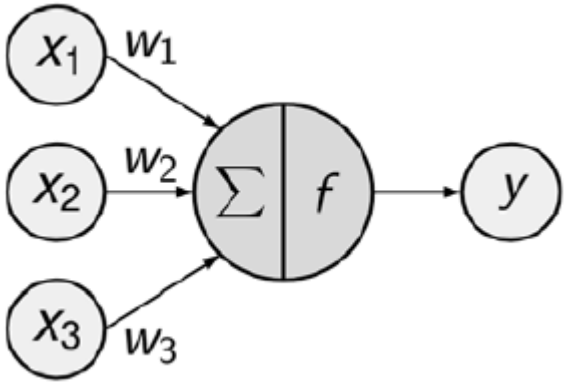
딥러닝

신경망 개요

- 생물학적 신경망
  - = network of interconnected cells
    - 85 billion neurons (human)
      - cf. 쥐 (75 million), 파리 (100k)
    - 궁극적 비교는 Turing test

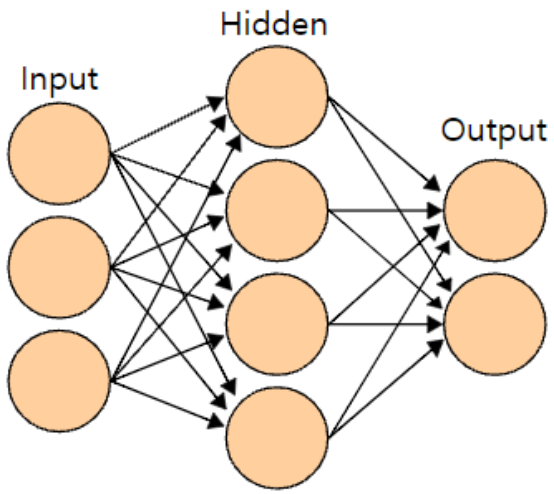


- 인공신경망
  - 생물학적 neuron → artificial nodes : 수신신호 → dendrites (수상돌기)



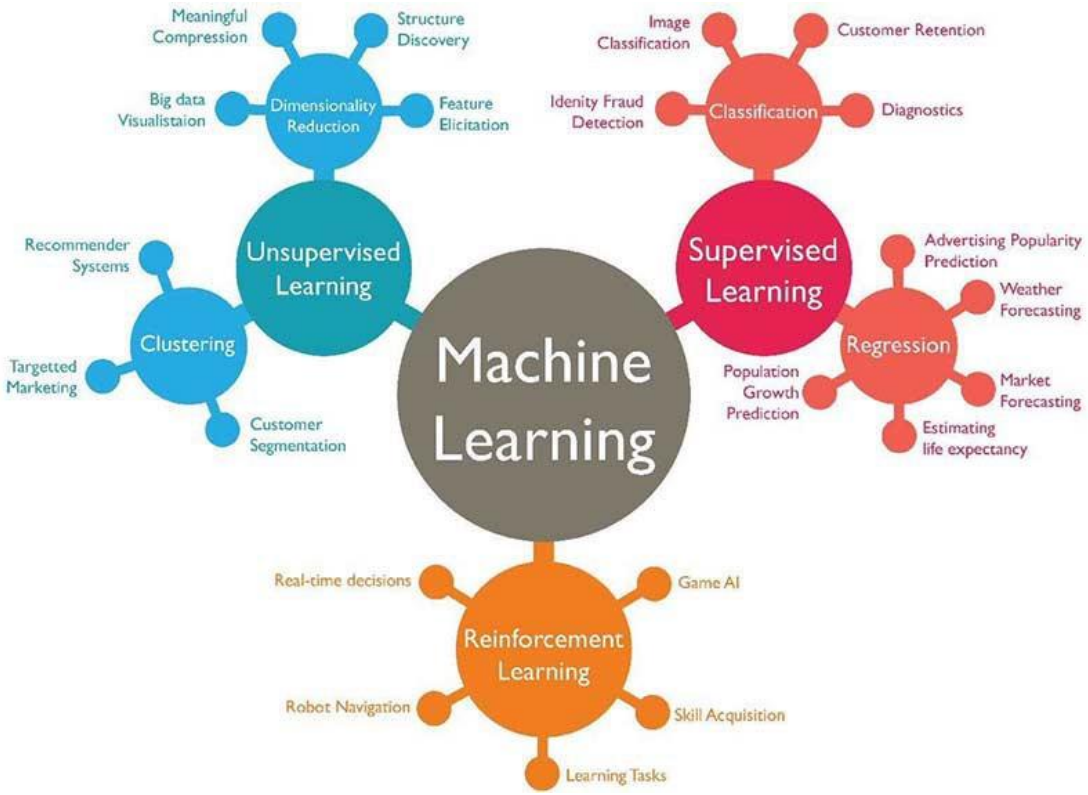
X var ≈ dendrite,
Σ = directed network diagram,
f = activation function

- 기본 개념
  - 선형 맞춤 (linear fitting)과 비선형 변환 (nonlinear transformation or activation)을 반복
  - 발전 과정
    - Backpropagation in 1980's
    - Kernel methods in early 2000's
    - Parameter training by unsupervised learning in 2006
    - CNN ; Data → Feature → Knowledge
    - RNN ; LTSM (Long-Short term memory)
- 개별 신경망의 특징을 결정하는 요소:
  - 활성함수
    - 입력 신호를 출력 신호로 처리
  - Network topology
  - Training 알고리즘
    - **Gradient descent**
    - Newton method, Conjugate gradient, …



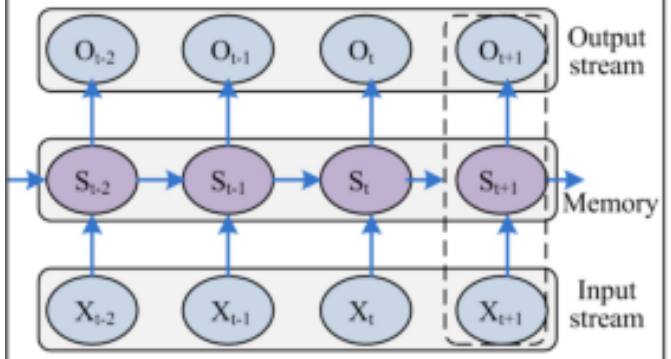
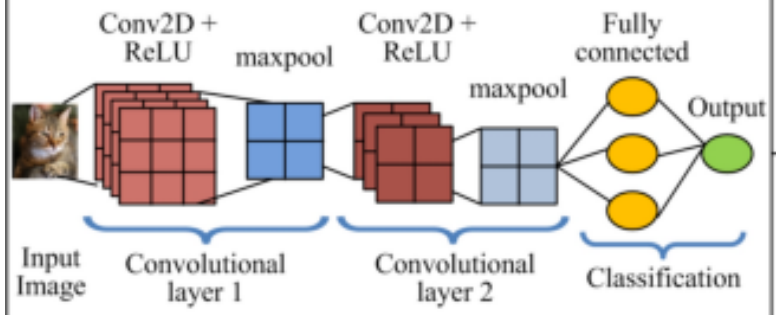
딥러닝 종류

- 감독형 딥러닝
  - CNN
  - RNN
- 무감독형 딥러닝
  - Autoencoder
  - RBM
- 강화학습
  - Q-Learning
  - Policy-Gradient

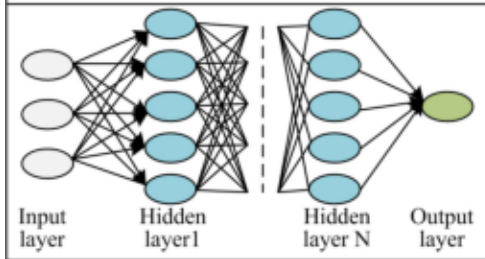
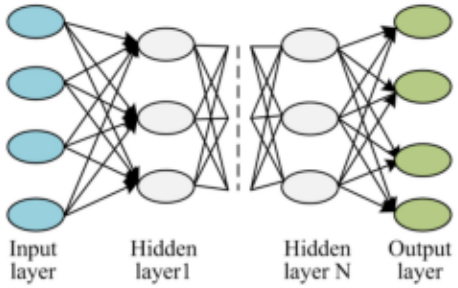
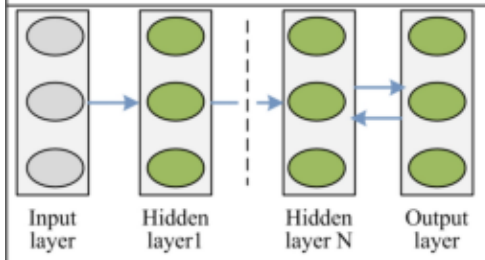
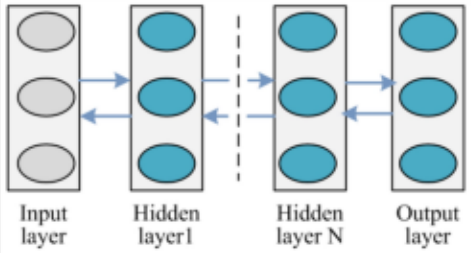


http://www.openwith.net

딥러닝 모델

Deep learning based model architecture	Brief description	Key features
	**Recurrent Neural Network (RNN)** • A neural network having capability of analyzing data stream • Share the same weight for each step. • Widely useful where output depends on previous results.	**Pros:** • Can memorize sequential events. • Good response in NLP area. **Cons:** • Learning issues due to gradient problem.
	**Convolutional Neural Network (CNN)** • This model is suitable for 2D data. • Designed by the functionality of human neuron's behavior. • Every hidden layer filter transform its input to 3D output volume for neuron activation..	**Pros:** • Few neuron connections required with respect to a typical NN. • So many variants have been proposed like AlexNet, GoogleNet etc. **Cons:** • It requires large labeled dataset.

https://link.springer.com/article/10.1007/s00530-020-00694-1

Deep learning based model architecture	Brief description	Key features
	**Deep Neural Network** • Generalized frame work of deep learning model. • Widely applicable for classification and regression model. • Consisting of more than 2-hidden layer.	**Pros:** • Widely used with high success rate in different area. **Cons:** • Learning process is very slow.
	**Deep Auto-encoder** • Mainly used for feature extraction and space dimensionality reduction. • It has same number of input and output nodes. • Type of unsupervised learning.	**Pros:** • No need of labeled data. • So many modified versions of auto-encoders are designed according to need. **Cons:** • Training can be suffered from errors.
	**Deep Belief Network** • Combination of restricted Boltzmann machine (RBM). • Hidden layer of each network is visible to the next layer. • It allows supervised and unsupervised learning.	**Pros:** • Used greedy learning approach to initialize the network. • Maximizing the likelihood inferences directly. **Cons:** • Training can be expensive due to initialization and sampling process.
	**Deep Boltzmann Machine** • Another proposed model of Boltzmann Machine family. • Possess conditionally independent connection between layers of network. • It uses stochastic MAX likelihood method to maximize the lower bound of likelihood..	**Pros:** • Integrate top-down feedback for robust inferences for indefinite inputs. **Cons:** • Higher time complexity for inference with respect to DBN. • Optimization of parameters is tedious for large dataset.

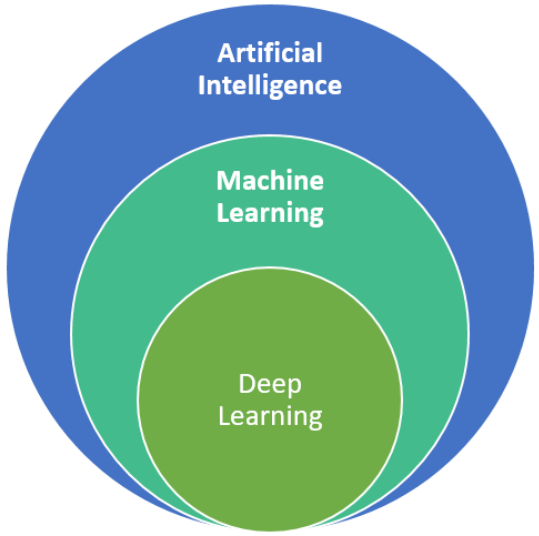
- 주요 딥러닝 프레임워크
  - 주의: 변화가 큼

Source: https://marutitech.com/top-8-deep-learning-frameworks/

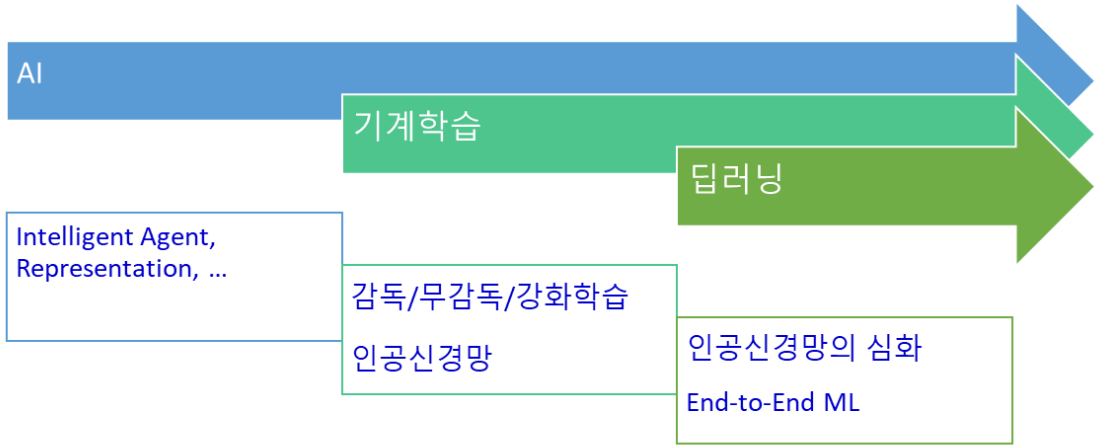
AI

AI와 데이터 과학

- AI
  - 개념
    - 인간의 학습능력, 추론능력, 지각능력을 인공적으로 구현하려는 컴퓨터 과학 세부분야 중 하나
    - https://ko.wikipedia.org/wiki/인공지능
  - *"Data is the Fuel for AI"*

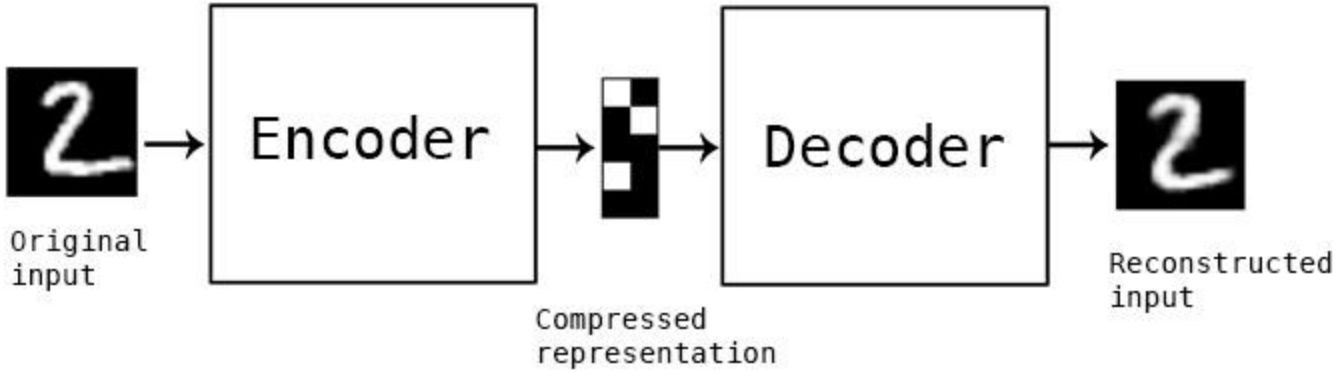


- 데이터과학
  - Domain 지식
  - Math/Statistics
  - 프로그래밍



예: 생성모델과 llm의 bio 활용

- 생성모델
  - 모델 훈련을 통해 주어진 데이터와 유사한 데이터를 (지정된 조건에 의해) 만들어 냄
    - Discriminative modeling estimates $p(y|x)$ vs. Generative modeling estimates $p(x)$.
    - Conditional generative models $p(x \mid y)$.

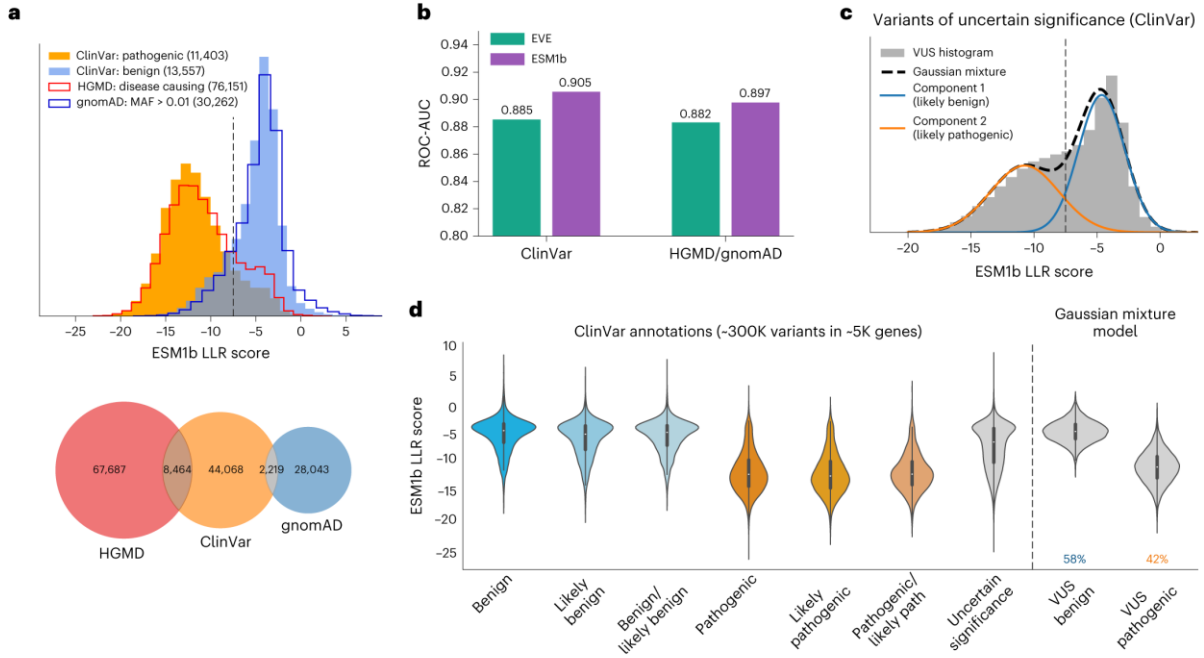


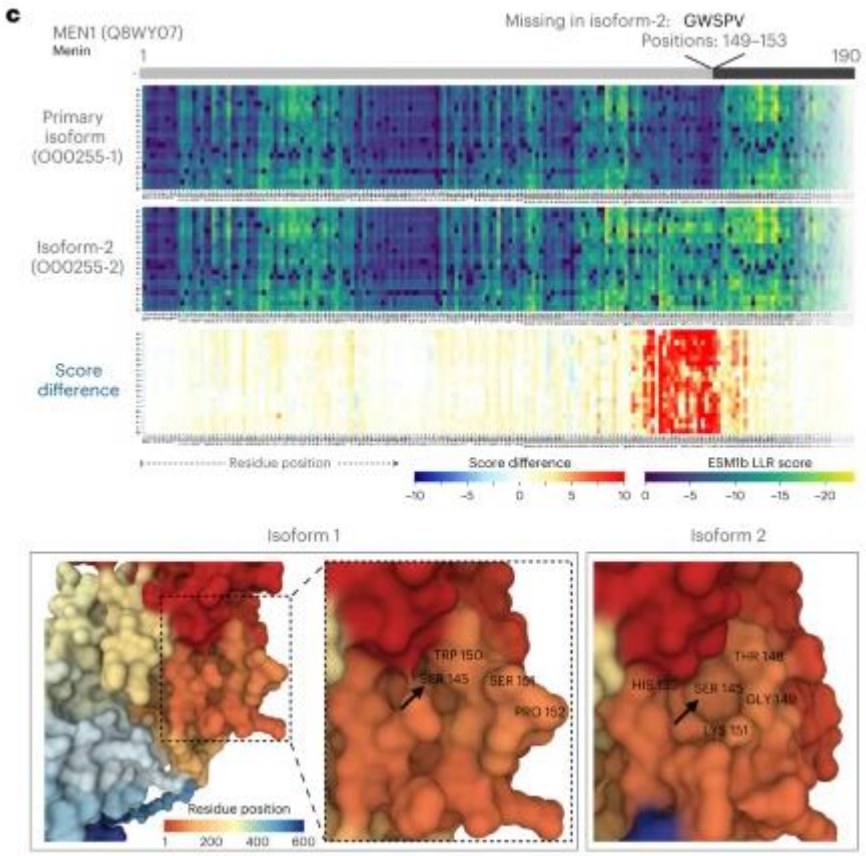
```
class Autoencoder(Model):
  def __init__(self, latent_dim, shape):
    super(Autoencoder, self).__init__()
    self.latent_dim = latent_dim
    self.shape = shape

    self.encoder = tf.keras.Sequential([
      layers.Flatten(),
      layers.Dense(latent_dim, activation='relu'), ])

    self.decoder = tf.keras.Sequential([
      layers.Dense(tf.math.reduce_prod(shape),
             activation='sigmoid'),
      layers.Reshape(shape)])

  def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded
```

```
shape = x_test.shape[1:]
latent_dim = 64
autoencoder = Autoencoder(latent_dim, shape)

autoencoder.compile(optimizer='adam',
loss=losses.MeanSquaredError())
--
autoencoder.fit(x_train, x_train,
                epochs=10,
                shuffle=True,
                validation_data=(x_test, x_test))
--
encoded_imgs = autoencoder.encoder(x_test).numpy()
decoded_imgs = autoencoder.decoder(encoded_imgs).numpy()
```

https://www.tensorflow.org/tutorials/generative/autoencoder

- LLM 활용 예: Genetics and genomics
  - exhaustive profiling of protein-disrupting variants across the genome
  - ← variants are linked to many protein alterations





ESM1b predicted effects of different SNP variant impacts on protein function. Figure from Brandes et al. demonstrates how a small splicing effect (excision of five amino acids from the primary isoform of the MEN1 protein) can result in dramatic changes in the predicted effects of variants in a much larger region. https://www.nature.com/articles/s41588-023-01465-0

수고
하셨습니다